SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Name : V.V.D.Nagasai Ratan Reddy	**Reg. no. 192324172**
Course Code: DSA03	**Course Name: Natural Language Processing**
Branch: AI&DS	**Year : 2023**

Assignment 5

1. **Write a finite-state automaton for a dialogue manager for checking your bank balance and withdrawing money at an automated teller machine.** (10M) (CO5) (BLT4)
2. **Consider the sentence "Christ opened the eyes", describes fact about English syntax.**
   a) Describe and illustrate how your favourite foreign language differs in this respect.
   b) Explain how a MT system could deal with this difference. (10M) (CO5) (BLT4)

Problem Overview and Dialogue Requirements

An automated teller machine (ATM) provides a spoken or text-based dialogue between a customer and a banking system. A dialogue manager controls the sequence of prompts, accepts user responses, checks whether each response is valid, and moves the interaction to the next appropriate state.

The required services are checking an account balance and withdrawing money. The dialogue must also handle supporting actions such as card insertion, PIN verification, transaction selection, amount entry, confirmation, cash dispensing, receipt printing, cancellation, and card ejection.

A finite-state automaton (FSA) is suitable because the interaction follows a finite set of states and transitions. Each state represents what the system is currently doing or expecting, while each transition is triggered by a user action or a system result.

Definition of a Finite-State Automaton

A deterministic finite-state automaton can be defined as a five-tuple $M=(Q, \Sigma, \delta, q0, F)$. Q is a finite set of states, Σ is the set of input symbols, δ is the transition function, q0 is the initial state, and F is the set of accepting or successful terminal states.

For an ATM, input symbols are dialogue events such as CARD_INSERTED, PIN_VALID, PIN_INVALID, SELECT_BALANCE, SELECT_WITHDRAWAL, AMOUNT_ENTERED, CONFIRM, CANCEL, and CASH_DISPENSED.

The FSA does not itself perform banking calculations. It controls the conversation and calls external banking services when necessary. For example, a balance enquiry state requests the balance from the bank server and then moves to a display state.

State Set for the ATM Dialogue

The main states are START, CARD_READ, PIN_ENTRY, PIN_CHECK, MAIN_MENU, BALANCE_REQUEST, BALANCE_DISPLAY, WITHDRAWAL_AMOUNT, WITHDRAWAL_CHECK, CONFIRM_WITHDRAWAL, CASH_DISPENSE, RECEIPT_OPTION, EJECT_CARD, and END. Additional error states are PIN_ERROR, INSUFFICIENT_FUNDS, INVALID_AMOUNT, NETWORK_ERROR, and CARD_RETAINED. These states allow the automaton to handle exceptional situations without losing control of the dialogue.

Each state has a defined purpose. PIN_ENTRY collects a PIN; PIN_CHECK validates it; MAIN_MENU asks the customer to select a service; WITHDRAWAL_CHECK checks account balance and withdrawal limits; and CASH_DISPENSE activates the cash dispenser only after successful authorization.

Input Alphabet and Events

The input alphabet Σ consists of events received from the customer interface or the banking system. Customer events include inserting a card, entering a PIN, choosing a menu option, entering an amount, confirming, cancelling, and selecting a receipt preference.

System events include CARD_ACCEPTED, PIN_VALID, PIN_INVALID, ACCOUNT_FOUND, BALANCE_RETURNED, FUNDS_SUFFICIENT, FUNDS_INSUFFICIENT, DISPENSE_SUCCESS, DISPENSE_FAILURE, and NETWORK_AVAILABLE.

Using named event symbols makes the transition table readable. Each edge in the state graph is labelled with the event that causes the transition. For example, PIN_ENTRY --SUBMIT_PIN--> PIN_CHECK.

Initial State and Card Insertion

The initial state is START. The ATM displays a welcome message and waits for a card. When the customer inserts a valid bank card, the system reads the chip or magnetic-stripe data and moves to CARD_READ.

If no card is inserted, the automaton remains in START. If the card is unreadable or unsupported, it displays an error and returns the card. A valid card leads to PIN_ENTRY.

The transition can be written as $\delta(\text{START},\text{CARD_INSERTED})=\text{CARD_READ}$ and $\delta(\text{CARD_READ},\text{CARD_ACCEPTED})=\text{PIN_ENTRY}$. A card-read failure can move to CARD_ERROR and then safely return to START.

PIN Entry and Authentication

In PIN_ENTRY, the ATM prompts the customer to enter a personal identification number. The customer submits the PIN, and the system sends the entered value to the bank's authentication service. The dialogue manager moves to PIN_CHECK.

If the PIN is valid, the system resets the invalid-attempt counter and moves to MAIN_MENU. If it is invalid, the system moves to PIN_ERROR, informs the customer, and allows another attempt subject to the bank's security policy.

After a fixed maximum number of failed attempts, commonly three in a simplified model, the card may be retained or the session blocked. This transition is represented as PIN_ERROR --MAX_ATTEMPTS--> CARD_RETAINED.

Main Menu and Transaction Selection

After authentication, the ATM displays the available services. The two required options are CHECK_BALANCE and WITHDRAW_CASH. Other options such as deposit, transfer, and mini-statement may be included as extensions.

If the customer selects balance enquiry, the automaton moves from MAIN_MENU to BALANCE_REQUEST. If the customer selects withdrawal, it moves to WITHDRAWAL_AMOUNT or a withdrawal setup state.

If the customer selects CANCEL or presses the cancel key, the automaton moves to EJECT_CARD. This ensures that a customer can safely end the session from the main menu.

Balance Enquiry Path

In BALANCE_REQUEST, the dialogue manager sends an authenticated account identifier to the banking server and requests the available balance. The ATM should not trust a locally stored balance because the bank's account system is authoritative.

If the server responds successfully, the system moves to BALANCE_DISPLAY and shows the available balance. The customer may then choose another transaction or finish the session.
If the network is unavailable or the server returns an error, the automaton moves to NETWORK_ERROR. It displays a suitable message and offers retry or safe termination. The account balance should not be fabricated or displayed from stale data without clear policy.

Withdrawal Amount Entry

In WITHDRAWAL_AMOUNT, the ATM asks the customer to enter the amount to withdraw. The amount must be positive, within daily limits, and compatible with the denominations supported by the machine.
An invalid amount, such as zero, a negative number, or an amount that cannot be dispensed, causes a transition to INVALID_AMOUNT. The customer is prompted to enter a different amount or cancel the transaction.
A valid amount moves the automaton to WITHDRAWAL_CHECK. The banking service verifies the available balance, withdrawal limits, and account status before authorization.

Funds Verification and Authorization

In WITHDRAWAL_CHECK, the system compares the requested amount with the available balance and applies withdrawal limits. It may also check whether the ATM has sufficient cash in the required denominations.
If the account has sufficient funds and the ATM can dispense the amount, the transaction moves to CONFIRM_WITHDRAWAL or directly to CASH_DISPENSE, depending on the interface design.
If funds are insufficient, the system moves to INSUFFICIENT_FUNDS and explains the problem. The customer may enter a smaller amount or return to the main menu. No cash is dispensed in this branch.

Confirmation and Cash Dispensing

CONFIRM_WITHDRAWAL displays the amount and asks the customer to confirm. If the customer confirms, the bank authorizes the debit and the ATM proceeds to CASH_DISPENSE. If the customer cancels, the transaction is aborted without dispensing cash.
The cash dispenser attempts to deliver the requested notes. A successful dispense produces DISPENSE_SUCCESS and moves to RECEIPT_OPTION. A hardware failure produces DISPENSE_FAILURE and triggers a reversal or reconciliation process.
Cash dispensing must occur only after authorization. The dialogue manager must not transition to a successful completion state merely because the user entered an amount. The system needs confirmation from the banking and dispenser components.

Receipt, Further Transaction, and Exit

After a balance enquiry or withdrawal, the ATM asks whether the customer wants a printed receipt. The receipt choice is represented by PRINT_RECEIPT or NO_RECEIPT events.

The customer may be offered another transaction. If YES is selected, the automaton returns to MAIN_MENU. If NO is selected, it moves to EJECT_CARD and then END after the card has been returned.

The terminal state END indicates that the session is complete. The machine should clear sensitive screen data, close the authenticated session, and return to its idle state.

Cancellation and Error Recovery

A robust dialogue manager must support cancellation from states where cancellation is safe. For example, the customer can cancel during PIN entry, menu selection, amount entry, and confirmation. The transition should lead to a safe cancellation state and card return.

Network errors, invalid inputs, and dispenser failures must have explicit paths. An error state should provide a clear message and a controlled choice: retry, return to menu, or terminate.

Not every error should return to the same state. A PIN error may allow another PIN attempt, while a dispenser error after a debit requires a bank reversal or manual reconciliation. The state design should reflect the operational consequences.

State Transition Table

A transition table lists the current state, input event, next state, and action. It provides a compact specification that can be used to implement the dialogue manager.

Example transitions: START + CARD_INSERTED → CARD_READ; CARD_READ + CARD_ACCEPTED → PIN_ENTRY; PIN_ENTRY + SUBMIT_PIN → PIN_CHECK; PIN_CHECK + PIN_VALID → MAIN_MENU; PIN_CHECK + PIN_INVALID → PIN_ERROR.

Further transitions: MAIN_MENU + SELECT_BALANCE → BALANCE_REQUEST; MAIN_MENU + SELECT_WITHDRAWAL → WITHDRAWAL_AMOUNT; BALANCE_REQUEST + BALANCE_RETURNED → BALANCE_DISPLAY; WITHDRAWAL_AMOUNT + VALID_AMOUNT → WITHDRAWAL_CHECK; WITHDRAWAL_CHECK + FUNDS_SUFFICIENT → CONFIRM_WITHDRAWAL.

Formal Transition Function

The transition function δ maps a state and an input symbol to a next state. For example, $\delta(\text{MAIN_MENU},\text{SELECT_BALANCE})=\text{BALANCE_REQUEST}$ and $\delta(\text{MAIN_MENU},\text{SELECT_WITHDRAWAL})=\text{WITHDRAWAL_AMOUNT}$.

For PIN handling, $\delta(\text{PIN_CHECK},\text{PIN_VALID})=\text{MAIN_MENU}$ and $\delta(\text{PIN_CHECK},\text{PIN_INVALID})=\text{PIN_ERROR}$. For withdrawal, $\delta(\text{WITHDRAWAL_CHECK},\text{FUNDS_SUFFICIENT})=\text{CONFIRM_WITHDRAWAL}$ and $\delta(\text{WITHDRAWAL_CHECK},\text{FUNDS_INSUFFICIENT})=\text{INSUFFICIENT_FUNDS}$.

For session completion, $\delta(\text{RECEIPT_OPTION},\text{FINISH})=\text{EJECT_CARD}$ and $\delta(\text{EJECT_CARD},\text{CARD_RETURNED})=\text{END}$. A practical implementation also includes a default transition for unexpected inputs, which displays an error and preserves a safe state.

ATM Dialogue FSA Diagram

The diagram below illustrates the major dialogue states and the paths for checking a balance and withdrawing cash. The top path covers card insertion, PIN validation, and menu selection. The balance branch displays the account balance, while the withdrawal branch collects an amount and dispenses cash after authorization.

The diagram is a simplified control-flow view. A production ATM would include additional states for card validation, encryption, host communication, cash inventory, receipt printing, timeouts, and reversal handling.

Every transition should have a clear event label. The diagram helps developers, testers, and analysts understand which user action or system response is required to move the interaction forward.

Figure 1. Finite-state dialogue flow for card authentication, balance enquiry, and cash withdrawal.

Security and Privacy Requirements

Security is essential in an ATM dialogue. PINs must be protected, never displayed in plain text, and never stored in application logs. The system should use secure communication with the bank and should terminate sessions after inactivity.

The dialogue manager should limit PIN attempts and handle card retention according to bank policy. It should also prevent a second transaction from using an already closed or unauthenticated session.

Privacy requirements include clearing the screen after the session, avoiding disclosure of sensitive account details to nearby people, and securely handling receipts. The state machine should include timeout and session-expiration transitions.

Testing the ATM Automaton

Each state and transition should be tested with normal, invalid, and boundary inputs. A successful balance test inserts a valid card, enters a correct PIN, chooses balance enquiry, receives a balance response, and ends the session.

A successful withdrawal test checks authentication, amount validation, sufficient funds, customer confirmation, and successful cash dispensing. Negative tests include incorrect PIN, excessive attempts, invalid amount, insufficient funds, network failure, and dispenser failure.

State-transition coverage measures whether each defined transition has been exercised. Path coverage checks complete transaction routes. These tests help verify that the dialogue manager does not skip authentication or dispense cash without authorization.

Limitations and Possible Extensions

A finite-state model is clear and reliable for a bounded dialogue, but the number of states can grow as more services and exceptions are added. Some states may need variables such as attempt_count, selected_account, amount, and transaction_status.

An extended finite-state machine (EFSM) can combine states with guards and actions. For example, a transition can be labelled PIN_INVALID [attempt_count < 3] / increment counter. This avoids creating a separate state for every possible attempt count.
Further extensions may include multilingual prompts, accessibility support, biometric authentication, contactless cards, cash deposits, fund transfers, and fraud detection. The basic FSA remains a useful foundation for these features.

Conclusion and Complete ATM FSA Summary

The ATM dialogue manager can be modeled as a finite-state automaton whose states represent dialogue stages and whose transitions are triggered by customer actions and banking-system responses.
The main path is START → CARD_READ → PIN_ENTRY → PIN_CHECK → MAIN_MENU. From the menu, the customer can select BALANCE_REQUEST or WITHDRAWAL_AMOUNT. A balance response is displayed, while a withdrawal is checked, confirmed, and dispensed only after authorization.
Error states handle invalid PINs, insufficient funds, invalid amounts, network failures, and hardware problems. The final EJECT_CARD → END path safely closes the session. The FSA provides a structured specification for implementing and testing the ATM dialogue.
A balance-enquiry trace follows START, CARD_READ, PIN_ENTRY, PIN_CHECK, MAIN_MENU, BALANCE_REQUEST, BALANCE_DISPLAY, EJECT_CARD, and END. A withdrawal trace follows the same authentication path, then WITHDRAWAL_AMOUNT, WITHDRAWAL_CHECK, CONFIRM_WITHDRAWAL, CASH_DISPENSE, RECEIPT_OPTION, and END.
These traces demonstrate that both services share the same authenticated session but branch according to the selected transaction. They also show why the automaton must include error handling and safe termination states.

Figure 2. Simplified transaction sub-automaton.

Introduction to English Syntax and Word Order

The sentence “Christ opened the eyes” is a simple English clause. It contains a subject, a verb, and a direct object. The subject is Christ, the verb is opened, and the object is the eyes.
English commonly follows Subject–Verb–Object (SVO) order in declarative sentences. The subject normally occurs before the finite verb, and the direct object normally follows the verb. Word order therefore provides important information about grammatical roles.
This question compares the English pattern with a foreign language and explains how a machine translation system can handle the difference. For illustration, Telugu is used as the comparison language because it commonly follows Subject–Object–Verb (SOV) order.

Constituent Analysis of the English Sentence

The sentence can be divided into a noun phrase and a verb phrase. The noun phrase “Christ” functions as the subject. The verb phrase “opened the eyes” contains the verb opened and the object noun phrase “the eyes”.

A simple phrase-structure representation is S → NP VP, VP → V NP, NP → ProperNoun or Det N. Applying these rules gives S [NP Christ] [VP [V opened] [NP [Det the] [N eyes]].

The structure shows that English places the verb before its object. The sequence is not arbitrary: “Christ the eyes opened” is not the normal unmarked English declarative order. Syntax determines how the words combine into a grammatical sentence.

English SVO Word-Order Pattern

English is generally classified as an SVO language. In a basic transitive clause, the subject precedes the verb and the object follows it. For example, “The child opened the door” follows the order The child (S), opened (V), the door (O).

The verb helps express tense and agreement, while the position of the noun phrases helps identify who performs the action and what receives it. English also uses prepositions before noun phrases, as in “in the room” and “with a key”.

English has flexibility in questions, passive constructions, and topicalization, but SVO is the usual order for simple affirmative clauses. A machine translation system must identify the underlying grammatical roles rather than copy the surface word order blindly.

Choosing Telugu as the Comparison Language

Telugu is used here as the example language. Telugu is commonly described as a Subject–Object–Verb (SOV) language. In a basic declarative clause, the subject appears first, the object follows, and the verb usually appears at the end.

A natural Telugu rendering of the meaning “Christ opened the eyes” can be expressed as “క్రీస్తు కళ్లను తెరిచాడు” (Krīstu kaḷḷanu tericāḍu), meaning Christ the-eyes opened. Exact wording can vary with context and the intended sense of “the eyes”.

The important syntactic difference is the order of the object and verb. English places opened before the eyes, whereas Telugu places the object phrase before the verb. The translation system must therefore reorder the constituents.

Telugu SOV Structure

In Telugu, the subject is generally followed by the object, and the verb occurs at the end of the clause. The example can be segmented as క్రీస్తు (Christ/subject), కళ్లను (eyes/object with accusative marking), తెరిచాడు (opened/verb).

The object marker -ను (-nu) helps identify the noun phrase as the object. Telugu uses case suffixes and postpositional or postposition-like constructions where English often uses prepositions. These grammatical markers help signal relations even when word order differs.

Because the object is morphologically marked, Telugu can sometimes allow variations in constituent order for emphasis. Nevertheless, SOV is the basic neutral pattern taught for simple declarative clauses.

Illustration: English Versus Telugu

The English sequence is Subject → Verb → Object: Christ → opened → the eyes. The Telugu sequence is Subject → Object → Verb: Christ → eyes-ACC → opened. The semantic roles remain the same even though the word order changes.

The following diagram compares the two sequences. It shows that machine translation must preserve the subject, action, and object while generating them in the target language's preferred order.

An MT system should not translate word by word in the original sequence. It should first identify the sentence structure, map each constituent to its target-language equivalent, and then generate the Telugu SOV order.

Figure 3. Comparison of English SVO and Telugu SOV word order.

Lexical and Morphological Differences

Translation involves more than changing word positions. English marks grammatical relations largely through word order and prepositions, while Telugu uses postpositions and case suffixes in many constructions.

The English definite article “the” does not have a direct one-to-one equivalent in every Telugu noun phrase. Telugu may express definiteness through context or other forms rather than using an obligatory article corresponding to English the.

The English past-tense verb opened must be rendered with an appropriate Telugu verb form. The target form may encode tense, aspect, person, number, and gender distinctions differently from English. A translation system must generate the correct morphology.

Constituent Structure and Phrase Reordering

A parser can represent the English input as S(NP,VP), with VP(V,NP). The transfer component identifies the subject NP, the verb V, and the object NP. It then applies the target-language order S(NP,NP, V).

The source tree has the structure [S [NP Christ] [VP [V opened] [NP the eyes]]]. A target-oriented structure can be represented as [S [NP Christ] [NP eyes-ACC] [VP opened].

This constituent-level transformation is safer than moving individual words without understanding their roles. It ensures that the object phrase remains together and that the verb is placed after its arguments.

How a Rule-Based MT System Handles the Difference

A rule-based machine translation system can use three main stages: analysis, transfer, and generation. Analysis parses the English sentence and identifies its grammatical constituents and features.

Transfer maps the English syntactic structure to a Telugu structure. It changes the order from SVO to SOV, maps the lexical items, and assigns target-language case marking. Generation then produces a grammatical Telugu sentence.

The system should preserve semantic roles. Christ must remain the subject and the eyes must remain the object. Only their surface placement and morphology change according to Telugu grammar.

Analysis Stage of Machine Translation

The analysis stage tokenizes the input and performs lexical lookup. It identifies Christ as a proper noun, opened as a past-tense transitive verb, the as a determiner, and eyes as a plural noun.

A syntactic parser builds a tree showing that Christ is the subject of opened and the eyes is its direct object. The parser may also identify tense and number features needed for target generation.

The analysis stage should resolve ambiguities where possible. Opened can have different meanings in different contexts, but in this sentence it is a transitive verb whose object is the noun phrase the eyes.

Figure 4. Main stages of a rule-based machine translation system.

Transfer Stage: Mapping English to Telugu

The transfer stage converts source-language structures into target-language structures. The English NP subject is mapped to a Telugu subject phrase. The English object NP is mapped to a Telugu object phrase with appropriate case marking.

The verb is mapped to a Telugu lexical verb and inflected for the required past-tense and agreement features. The target structure places the object before the verb, producing SOV order.

The definite article the may be omitted or handled contextually because Telugu does not require an exact article equivalent in every noun phrase. The transfer rules must use target-language grammar rather than force English function words into Telugu.

Generation Stage: Producing Telugu

Generation selects the Telugu word forms and arranges them according to the target syntax. The subject phrase comes first, the object phrase follows, and the finite verb appears at the end.

An illustrative output is “క్రీస్తు కళ్లను తెరిచాడు”. The output preserves the meaning that Christ performed the action of opening the eyes. The accusative marker on the object helps identify its grammatical role.

A high-quality system should also check agreement and naturalness. The selected verb form must agree with the subject and convey the intended tense. Human review or a target-language language model can improve fluency.

Statistical and Neural MT Approaches

Modern statistical machine translation learns translation correspondences and reordering patterns from parallel bilingual corpora. It can learn that English verb-object sequences often correspond to Telugu object-verb sequences.

Neural machine translation uses an encoder-decoder architecture, often with attention or a Transformer. The encoder represents the source sentence, and the decoder generates target tokens in an order learned from bilingual examples.

Although neural systems may learn reordering without explicit hand-written rules, the underlying challenge remains the same: the system must preserve semantic roles while producing target-language word order and morphology.

Alignment and Semantic Roles

Word alignment connects source-language words or phrases to their target-language equivalents. In the example, Christ aligns with క్రీస్తు, the eyes aligns with కళ్లను, and opened aligns with తెరిచాడు.

Semantic roles identify the participant performing the action and the entity affected by it. Christ is the agent and the eyes are the patient or theme. These roles must remain stable even when the word sequence changes.

An MT system that relies only on positional copying could incorrectly generate an SVO sequence in Telugu. Syntactic parsing and semantic-role information help prevent this error.

Handling Articles, Number, and Case

The English phrase the eyes contains a determiner and a plural noun. Telugu may express the same reference without a separate article, depending on context. The translator must decide whether the definite meaning is contextually recoverable.

The object form in Telugu may use accusative marking. This morphological marking is important because it identifies the object and supports flexible constituent ordering. The target generator should apply the appropriate suffix to the noun phrase.

Number and agreement must also be considered. English plural eyes may have a target-language equivalent whose number marking and lexical form depend on the context. Translation is therefore a combination of lexical selection, morphology, and syntax.

Difficulties in Direct Word-for-Word Translation

A word-for-word system might produce a Telugu sentence with the English order Subject–Verb–Object. Such an output may be understandable in some contexts but would not be the normal neutral Telugu structure.

Direct translation may also mishandle articles, case suffixes, and verb inflection. English uses a separate determiner before the noun, whereas Telugu can express grammatical relations through suffixes and context.

The solution is structural translation. The system should identify constituents, map grammatical functions, reorder phrases, and generate target morphology. This is the central lesson of the example.

Evaluation of the Machine Translation Output

The output can be evaluated for adequacy and fluency. Adequacy asks whether the target sentence preserves the meaning of the source. Fluency asks whether the target sentence is grammatical and natural in Telugu.

Human evaluators can check whether the subject, action, object, tense, and intended meaning are preserved. Automatic metrics such as BLEU can compare generated translations with reference translations, although they do not fully measure meaning.

Errors can be classified as lexical errors, word-order errors, morphology errors, omission, or addition. This classification helps developers improve the analysis, transfer, or generation stage responsible for the error.

Comparison with Other SOV Languages

Telugu is not the only language with a basic SOV order. Japanese, Korean, Hindi, and Turkish are commonly described as SOV languages, although each has its own case marking, agreement, and word-order flexibility.

English prepositions often correspond to postpositions or case markers in SOV languages. The verb-final structure means that a translator may need to delay the main verb until the end of the target clause.

These cross-linguistic patterns show why machine translation requires language-specific grammar. A single universal word-order rule cannot correctly generate every target language.

Summary of Syntax Difference and MT Solution

English generally uses SVO order in a simple declarative sentence: “Christ opened the eyes.” Telugu commonly uses SOV order, giving an illustrative rendering “క్రీస్తు కళ్లను తెరిచాడు”.

The syntactic difference is that English places the verb before the object, whereas Telugu places the object before the verb. Telugu also uses case marking and does not require a direct equivalent of the English definite article in every context.

A machine translation system handles the difference by parsing the English sentence, identifying subject, verb, and object, transferring the grammatical roles to Telugu, reordering the constituents into SOV, generating case and verb morphology, and checking the output for fluency.

Worked Translation Walkthrough and Final Comparison

Begin with the English parse: the complete sentence is an S node, its subject is the NP Christ, and its predicate is a VP containing the verb opened followed by the object NP the eyes. This parse identifies the grammatical roles before any translation takes place.

Next, transfer the roles into Telugu. Keep Christ as the subject, map the eyes to an object phrase with suitable accusative marking, and select the Telugu past-tense verb form. Arrange the target constituents as subject, object, and verb rather than copying the English sequence. The resulting illustrative sentence is “క్రీస్తు కళ్లను తెరిచాడు”. The translation preserves who performed the action and what was affected. The main lesson is that MT must transfer grammatical structure and meaning, not merely replace each English word with a dictionary equivalent.